

Análisis comparativo de modelos de aprendizaje de máquina para la detección de malaria en imágenes microscópicas de células sanguíneas.

Juan José Garzón Mejía y Juan José Barrientos Salazar

*Systems Engineering 504, Faculty of Engineering, Department of System Engineering
Universidad de Antioquia (UdeA), Calle 50 No. 73–21, 050034, Medellín, Colombia
juan.garzonm@udea.edu.co, juanj.barrientos@udea.edu.co*

Abstract—Este es el abstract del proyecto, se generará cuando se termine el último entregable junto con la propuesta del mismo y los resultados obtenidos.

Index Terms—Aprendizaje de máquina, clasificación binaria, malaria, redes neuronales convolucionales, visión por computador.

I. INTRODUCCIÓN

LA malaria constituye una de las enfermedades infecciosas de mayor impacto en la salud pública global [1], siendo causada por parásitos del género *Plasmodium* y transmitida principalmente mediante la picadura del mosquito *Anopheles*. A pesar de la existencia de protocolos diagnósticos consolidados, el análisis microscópico de frotis sanguíneos permanece como el estándar de oro, especialmente en entornos de recursos limitados donde se requiere un balance entre bajo costo y efectividad [2]. No obstante, la fiabilidad de este procedimiento se ve condicionada por la pericia del observador, la fatiga diagnóstica y la variabilidad inter-especialista, factores que han impulsado la investigación de soluciones basadas en el aprendizaje de máquina para automatizar la identificación de eritrocitos infectados [3], [4].

En el presente proyecto se aborda el desarrollo de un sistema de detección de malaria fundamentado en el procesamiento de imágenes digitales de células sanguíneas. El conjunto de datos empleado se define mediante una tarea de clasificación binaria supervisada, donde cada muestra es categorizada como parasitada o no parasitada. A diferencia del análisis de datos tabulares convencionales, la información de entrada reside en matrices de píxeles que encapsulan la morfología y textura celular, lo que demanda modelos capaces de aprender jerarquías de características visuales de manera autónoma [5].

Desde la perspectiva del análisis exploratorio de datos, el dataset permite evaluar dimensiones críticas como la distribución de clases, la resolución de las imágenes y la variabilidad en las condiciones de adquisición (iluminación, contraste y nitidez). El manejo del desbalance de clases y el ruido visual derivado de la tinción de las muestras son retos inherentes al dominio de la visión por computador aplicada a la medicina [2]. Estos factores resultan determinantes para garantizar la

robustez del modelo frente a variaciones biológicas y técnicas en entornos clínicos reales.

La integración de modelos de aprendizaje profundo en este contexto ofrece una oportunidad para optimizar el flujo de trabajo en los laboratorios de hematología, actuando como un sistema de apoyo a la toma de decisiones que prioriza casos críticos y reduce la carga operativa [6]. Académicamente, este problema facilita el estudio comparativo entre paradigmas de modelado, abarcando desde la ingeniería de características manual hasta la extracción automática mediante arquitecturas profundas [7].

La metodología propuesta se enmarca en el aprendizaje supervisado, permitiendo contrastar el desempeño de modelos clásicos —como Support Vector Machines (SVM) o Random Forest, alimentados por descriptores visuales— frente a Redes Neuronales Convolucionales (CNN) que procesan la información de forma end-to-end. Diversas investigaciones han demostrado que el uso de arquitecturas como ResNet o MobileNet puede ofrecer una ventaja significativa en la precisión diagnóstica bajo estas condiciones [8].

Finalmente, la estrategia experimental contempla la partición estratificada de los datos y la implementación de técnicas de aumento de datos (*data augmentation*) para mitigar el sobreajuste. Dado el carácter clínico de la aplicación, el rendimiento del sistema se evalúa mediante un análisis exhaustivo de la matriz de confusión y métricas de sensibilidad, especificidad y el área bajo la curva (AUC), garantizando así que el modelo no solo sea preciso, sino también confiable para la detección de casos positivos en entornos de tamizaje médico.

II. TRABAJO RELACIONADO

La aplicación de técnicas de visión por computador y aprendizaje automático en el análisis de imágenes médicas ha sido un área de intensa investigación, demostrando la viabilidad de utilizar algoritmos para la detección de anomalías celulares y sentando las bases para su aplicación en enfermedades infecciosas como la malaria [5].

En las primeras aproximaciones computacionales a este problema, la literatura destaca el uso de métodos de aprendizaje automático clásico. Estos enfoques dependían de la extracción manual de características (*feature engineering*),

donde se calculaban métricas de color, forma y textura de las células antes de clasificar los datos utilizando algoritmos como Máquinas de Vectores de Soporte (SVM) o *K-Nearest Neighbors* (k-NN). Aunque estos modelos resultaban efectivos en entornos controlados, presentaban limitaciones para generalizar frente a variaciones en la iluminación o en la preparación de las muestras, lo que motivó la búsqueda de representaciones más robustas [9].

Con la maduración del aprendizaje profundo (*Deep Learning*), el paradigma se desplazó hacia el uso de Redes Neuronales Convolucionales (CNN), las cuales automatizan la jerarquización y extracción de características visuales desde los píxeles crudos. Trabajos como los de Segovia *et al.* [4] y Rojas [3] han evidenciado que las CNN pueden identificar patrones sutiles del parásito *Plasmodium* con alta precisión, superando con creces a los métodos tradicionales. Asimismo, se han propuesto arquitecturas avanzadas para manejar el ruido visual inherente a las imágenes microscópicas y mejorar la generalización del modelo [7].

En el contexto de la optimización y aplicabilidad práctica, la literatura reciente se ha enfocado en evaluar qué arquitecturas ofrecen el mejor balance entre precisión y eficiencia computacional. Por ejemplo, Bautista *et al.* [8] realizaron un análisis comparativo entre modelos como ResNet-50V2, MobileNetV2 y EfficientNetB0, destacando cómo los modelos más ligeros pueden ser implementados en hardware de bajos recursos sin sacrificar significativamente el rendimiento diagnóstico. De igual manera, se ha buscado mejorar la interpretabilidad de estas redes para generar mayor confianza en su uso en entornos clínicos [6].

Finalmente, los enfoques más integrales no solo abordan la clasificación binaria de la célula, sino también la evaluación de la calidad de las muestras en los frotis sanguíneos teñidos [2]. Esto resulta fundamental para evitar falsos positivos derivados de artefactos de tinción o suciedad en el portaobjetos. El presente proyecto toma como base esta transición desde el aprendizaje clásico hacia el profundo, proponiendo una evaluación metodológica de diferentes modelos aplicados a la detección de malaria.

III. METODOLOGÍA

El estudio aborda la clasificación binaria de imágenes de frotis sanguíneo en dos clases — *Parasitized* (células infectadas por *Plasmodium*) y *Uninfected* (sanas) — mediante una arquitectura de dos etapas: primero un encoder convolucional entrenado con pérdida contrastiva supervisada (SupCon) [10], y luego cinco clasificadores estadísticos clásicos entrenados sobre los embeddings producidos por dicho encoder. Esta separación entre aprendizaje de representaciones y clasificador final permite una comparación justa de los algoritmos sobre un mismo espacio de características, aislando el aporte del entrenamiento contrastivo frente al del clasificador.

A. Dataset y preparación

Se utilizó el dataset público *Cell Images for Detecting Malaria* disponible en Kaggle [11], compuesto por 27 558

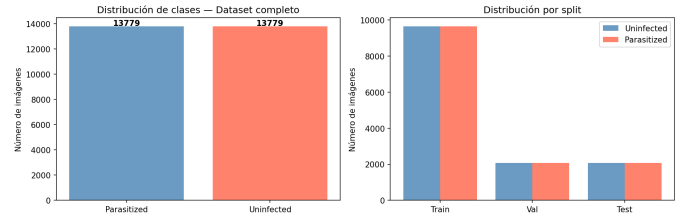


Fig. 1. Distribución de las dos clases en el dataset completo y dentro de cada partición. El balance perfecto 50/50 se preserva tras la estratificación.

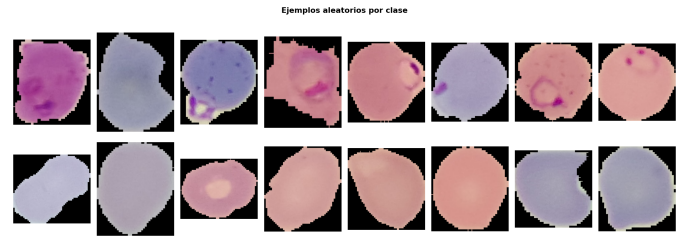


Fig. 2. Muestras representativas por clase. La diferencia visual principal es la presencia de la mancha violácea del *Plasmodium* en las células infectadas.

imágenes RGB de células sanguíneas adquiridas con microscopía óptica, distribuidas de forma equilibrada entre las dos clases (13 779 *Parasitized* y 13 779 *Uninfected*). La etiqueta binaria es $y \in \{0, 1\}$ con $0 = \text{Uninfected}$ y $1 = \text{Parasitized}$.

Las imágenes presentan variabilidad natural en tamaño (rango aproximado 80×80 a 200×200 píxeles). Todas se redimensionaron a un tamaño canónico de 96×96 píxeles, y se normalizaron canal a canal usando las estadísticas estándar de ResNet:

$$\mu = (0.485, 0.456, 0.406), \quad \sigma = (0.229, 0.224, 0.225). \quad (1)$$

El conjunto se dividió mediante muestreo estratificado por clase (semilla $\text{seed} = 42$) en tres particiones: *train* (70 %, 19 290 imágenes), *val* (15 %, 4 133 imágenes) y *test* (15 %, 4 135 imágenes). La estratificación garantiza balance exacto 50/50 en cada split, eliminando la necesidad de técnicas de remuestreo y permitiendo interpretar la accuracy global como una métrica no sesgada.

La Figura 1 ilustra cómo el balance perfecto se conserva en cada partición tras el muestreo estratificado, condición fundamental para la validez del entrenamiento supervisado.

Para evidenciar la naturaleza visual del problema, la Figura 2 muestra ejemplos representativos de ambas clases, donde la diferencia diagnóstica fundamental es la presencia de la mancha violácea correspondiente al parásito *Plasmodium* en las células infectadas.

La heterogeneidad de las resoluciones originales se examina en la Figura 3, la cual justifica la necesidad de redimensionar las muestras a un tamaño canónico para garantizar tensores de entrada homogéneos al encoder.

El análisis cromático del dataset, presentado en la Figura 4, evidencia un predominio del canal rojo, consistente con la tinción de Giemsa empleada en la preparación de frotis sanguíneos. Esta característica condiciona el diseño de las

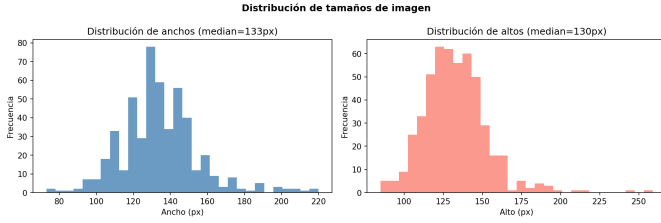


Fig. 3. Histograma 2D (ancho × alto) sobre una muestra aleatoria de 500 imágenes. La heterogeneidad original justifica el resize a 96×96 .

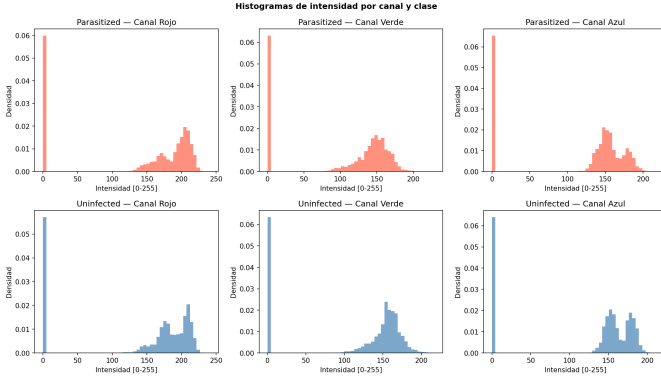


Fig. 4. Histogramas de intensidad por canal R, G, B. Predominio del canal rojo (eritrocitos teñidos con Giemsa).

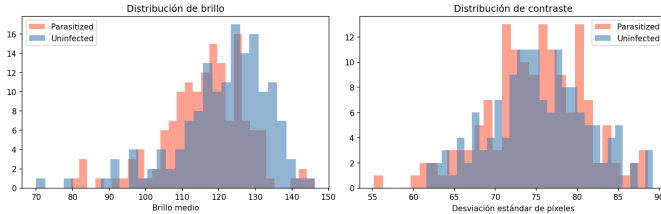


Fig. 5. Distribuciones de brillo medio y contraste (desviación estándar de intensidad) por clase. Las distribuciones se solapan, lo que indica que la discriminación no puede apoyarse únicamente en estadísticos globales.

augmentaciones para no alterar la información cromática diagnóstica.

Adicionalmente, la Figura 5 expone que las distribuciones de brillo y contraste por clase se solapan considerablemente, lo que descarta soluciones basadas únicamente en estadísticos globales de intensidad y motiva el uso de representaciones aprendidas mediante redes profundas.

B. Aumentaciones para entrenamiento contrastivo

Las técnicas de *data augmentation* desempeñan un rol fundamental en la robustez y generalización de modelos profundos de visión [12], particularmente en regímenes de aprendizaje auto-supervisado y contrastivo donde las vistas estocásticas constituyen la fuente principal de señal. Para el entrenamiento del encoder se aplicó una cadena de aumentaciones estocásticas que, por cada imagen x , genera dos vistas independientes v_1, v_2 , produciendo un batch efectivo de $2N$ muestras a partir de N imágenes originales. La cadena incluye `RandomResizedCrop` con escala uniforme en $[0.7, 1.0]$, `RandomHorizontalFlip` y

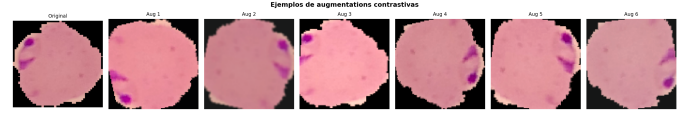


Fig. 6. Seis vistas independientes generadas a partir de una misma imagen mediante la cadena de aumentaciones contrastivas. Todas conservan la presencia (o ausencia) del parásito.

`RandomVerticalFlip` ($p = 0.5$), `RandomRotation` en $[-15^\circ, +15^\circ]$, `ColorJitter` con *brightness*, *contrast* y *saturation* de 0.2 (manteniendo *hue* fijo para preservar el color violáceo del parásito), `GaussianBlur` aplicado con probabilidad 0.3, y finalmente `ToTensor` más `Normalize`.

Se omitieron deliberadamente operaciones de oclusión agresiva (*Cutout*, *GridMask*, *RandomErasing*) porque pueden borrar accidentalmente la región del parásito —la entidad diagnóstica— y degradar la señal supervisada. La Figura 6 muestra seis vistas generadas a partir de una misma imagen, todas conservando la presencia del parásito.

C. Arquitectura del encoder contrastivo

El modelo `ContrastiveEncoder` está compuesto por tres bloques secuenciales: un *backbone* ResNet18 [13] preentrenado en ImageNet [14] (con la capa `fc` reemplazada por `nn.Identity`, exponiendo $h \in \mathbb{R}^{512}$), un bloque `encoder_head` que expande la representación a $z \in \mathbb{R}^{1024}$ mediante una capa lineal seguida de `BatchNorm1d` y `ReLU`, y un `proj_head` con dos capas lineales ($1024 \rightarrow 512 \rightarrow 128$) seguido de normalización L2, que produce $p \in S^{127}$ (esfera unidad en $\mathbb{R}^{128}$).

El bloque `proj_head` se emplea únicamente durante el cómputo de la pérdida contrastiva —la normalización L2 convierte el producto interno en similitud coseno— y se descarta tras el entrenamiento. La elección de expandir el embedding *downstream* a una dimensionalidad superior (1024) a la del *backbone* (512) sigue las recomendaciones empíricas reportadas en la literatura de aprendizaje contrastivo [10], [15], según las cuales la representación previa al *projection head* es más rica que la salida directa del *backbone*.

D. Función de pérdida: Supervised Contrastive Loss

La pérdida contrastiva supervisada implementada corresponde a la formulación propuesta por Khosla *et al.* [10]:

$$\mathcal{L}_{\text{SupCon}} = \sum_{i \in I} \frac{-1}{|P(i)|} \sum_{p \in P(i)} \log \frac{\exp(z_i \cdot z_p / \tau)}{\sum_{a \in A(i)} \exp(z_i \cdot z_a / \tau)} \quad (2)$$

donde $I = \{1, \dots, 2N\}$ es el conjunto de índices del batch multi-vista, $A(i) = I \setminus \{i\}$ es el conjunto de anclas sin el propio índice, $P(i) = \{p \in A(i) : y_p = y_i\}$ es el conjunto de positivos respecto al ancla i , $z_i \in S^{127}$ es la proyección L2-normalizada, y τ es la temperatura, fijada a $\tau = 0.07$. Intuitivamente, la pérdida maximiza la similitud coseno entre el ancla y todas sus contrapartes positivas, relativa a la similitud frente al resto del batch.

Su implementación se realiza con estabilización numérica explícita: cálculo de la matriz de similitud escalada $S =$

ZZ^T/τ , aplicación de máscaras booleanas para excluir auto-similitudes y aislar positivos, sustracción del máximo por fila (*log-sum-exp trick*) y promedio sobre positivos por fila y promedio global.

E. Procedimiento de entrenamiento del encoder

El entrenamiento se realiza con optimizador AdamW [16] (*learning rate* inicial 10^{-3} , *weight decay* 10^{-4}), *scheduler* CosineAnnealingLR [17] con $\eta_{\min} = 10^{-5}$, hasta 50 épocas y batch size 256 en GPU NVIDIA Tesla T4. Se emplea precisión mixta (`torch.cuda.amp`) en PyTorch [18] para reducir uso de VRAM y se aplica *early stopping* con paciencia de 7 épocas sobre la pérdida de validación. Por cada índice se obtiene un triplete (v_1, v_2, y) y las dos vistas se concatenan a lo largo de la dimensión del batch antes del *forward*. Se persisten dos checkpoints: `encoder_best.pt` (mejor `val_loss`) y `encoder_last.pt` (última época).

F. Extracción de embeddings

Tras la convergencia del encoder, los embeddings de las tres particiones se extraen *offline* mediante `model.get_embedding`, función que ejecuta el *forward* hasta `encoder_head` y descarta `proj_head`. La salida es un tensor $z \in \mathbb{R}^{1024}$ sin normalizar, con valores no negativos consistentes con la activación ReLU final (rango aproximado $[0, 8.875]$). Los embeddings se persisten como arrays NumPy `float32`: $X_{\text{train}} \in \mathbb{R}^{19290 \times 1024}$, $X_{\text{val}} \in \mathbb{R}^{4133 \times 1024}$ y $X_{\text{test}} \in \mathbb{R}^{4135 \times 1024}$.

G. Modelos clásicos sobre embeddings

Sobre los embeddings de 1024 dimensiones se entrenan cinco clasificadores estadísticos implementados en *scikit-learn* [19], todos envueltos en un `Pipeline(StandardScaler  $\rightarrow$  modelo)`. La búsqueda de hiperparámetros se realiza con `GridSearchCV` (3 *folds* estratificados, *scoring* `f1_macro`) sobre $X_{\text{trainval}} = X_{\text{train}} \cup X_{\text{val}}$.

Los cinco modelos son: (a) Regresión logística con regularización L2; (b) *k-Nearest Neighbors* [20] con distancia coseno; (c) Random Forest [21] con *bagging* de árboles de decisión; (d) MLP de una capa oculta con 256 unidades, activación ReLU y *early stopping* interno; y (e) Support Vector Machine [22] con kernel lineal. Al desactivar `probability=False` en el SVM, este último no produce *scores* probabilísticos, por lo que no se reporta AUC para dicho modelo.

H. Reducción de dimensión y análisis de features

PCA. El análisis de componentes principales se aplica sobre X_{train} centrado, seleccionando el menor k tal que la varianza acumulada supere el 95 %. Los embeddings se proyectan a $\mathbb{R}^k$ y los dos mejores modelos (regresión logística y SVM) se reevalúan sobre este espacio reducido.

UMAP. *Uniform Manifold Approximation and Projection* [23] construye un grafo k -NN ponderado en el espacio original y aprende un *embedding* de baja dimensión que aproxima la distribución de vecindades, con parámetros $n_{\text{neighbors}} = 15$, `min_dist` = 0.1, métrica coseno y $d = 2$ para visualización.

Análisis de features individuales. Para identificar qué dimensiones son más discriminativas, se computa la correlación punto-biserial r_{pb} entre cada *feature* y la etiqueta binaria, combinándola con la varianza individual para producir un *ranking* de las 50 dimensiones más informativas.

I. Métricas y validación estadística

Para cada modelo se reportan *accuracy*, *balanced accuracy*, F_1 -macro y F_1 -weighted, *precision-macro*, *recall-macro*, ROC-AUC [24] (cuando aplica) y la matriz de confusión 2×2 . Para el conjunto de test se calcula adicionalmente un intervalo de confianza bootstrap percentil al 95 % [25], mediante $B = 200$ resamplios con reemplazo, fijando `random_state` = 42. Esto permite cuantificar la incertidumbre de las métricas y comparar modelos estadísticamente.

Para evaluar la calidad geométrica de los embeddings se calculan las distribuciones de similitud coseno intra-clase e inter-clase sobre un subconjunto aleatorio de hasta 500 muestras del test, resumiéndolas con el *gap* de separabilidad $\Delta = \mu_{\text{intra}} - \mu_{\text{inter}}$.

J. Reproducibilidad

Todas las fuentes de aleatoriedad se fijan globalmente mediante una función `set_global_seed(seed = 42)` que afecta a `random`, `numpy`, `torch` (incluyendo CUDA), la variable de entorno `PYTHONHASHSEED`, y configura `torch.backends.cudnn` en modo determinista. Los splits resultantes se versionan en `data/processed/` y los hiperparámetros en archivos YAML versionados.

IV. RESULTADOS

A. Caracterización del dataset

El análisis exploratorio confirma las propiedades requeridas para un entrenamiento supervisado robusto: cardinalidad total de 27 558 imágenes con balance exacto 50/50, integridad completa del dataset (cero archivos corruptos), particiones estratificadas que conservan el balance en cada split, y variabilidad visual no trivial (Figura 5) que descarta soluciones basadas únicamente en estadísticos globales y motiva el uso de representaciones aprendidas.

B. Entrenamiento contrastivo

El encoder ResNet18 con `encoder_head` y `proj_head` se programó para entrenarse durante 50 épocas en una GPU NVIDIA Tesla T4 con batch efectivo de $2 \cdot 256 = 512$ vistas por paso. La Figura 7 muestra la trayectoria de la pérdida SupCon en *train* y validación.

El mejor checkpoint (`encoder_best.pt`, 147.7 MB) corresponde a la época 30 con `val_loss` = 5.5883. A partir de ese punto la pérdida de validación deja de mejorar y el *early stopping* termina la ejecución antes de las 50 épocas, lo que sugiere que el encoder ha convergido a un mínimo estable.

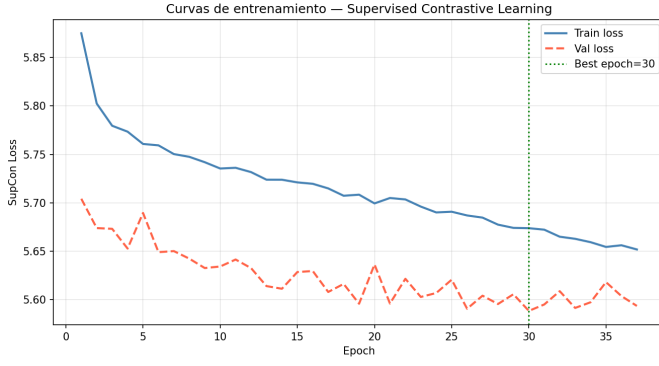


Fig. 7. Pérdida SupCon en *train* y *val* por época durante el entrenamiento del encoder.

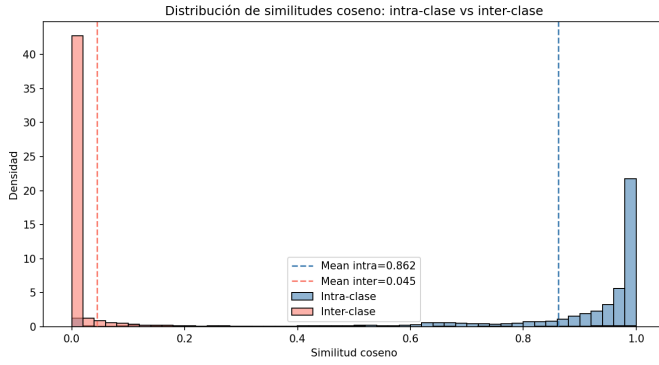


Fig. 8. Histogramas de similitud coseno intra-clase (mismo y) vs. inter-clase (distinto y) en el espacio de embeddings de 1024 D.

C. Calidad geométrica de los embeddings

Tras descartar `proj_head`, los embeddings de 1024 dimensiones exhiben una estructura altamente discriminativa. La Figura 8 muestra las distribuciones de similitud coseno intra-clase e inter-clase sobre una muestra aleatoria de 500 imágenes del test.

Los estadísticos resumidos son $\mu_{\text{intra}} = 0.8616$ ($\sigma = 0.2382$) y $\mu_{\text{inter}} = 0.0446$ ($\sigma = 0.1605$). El *gap* de separabilidad es por lo tanto $\Delta = 0.8170$, valor que indica una separación marcada entre clases bajo la métrica coseno. Esta observación geométrica se refuerza visualmente en la Figura 9, donde los bloques diagonales (intra-clase) aparecen uniformemente cercanos a 1, mientras que los bloques fuera de la diagonal (inter-clase) son visiblemente más oscuros.

El análisis de discriminabilidad por dimensión identifica un subconjunto reducido de *features* especialmente informativas, con índices Top-5 [269, 729, 827, 208, 625] y una discriminabilidad media de 0.8538 sobre las 50 mejores dimensiones. La Figura 10 grafica el *score* de discriminabilidad de las dimensiones más informativas, evidenciando que la información de clase está concentrada en pocas direcciones del espacio latente.

D. Reducción dimensional

Al aplicar PCA sobre X_{train} se observa que la varianza se concentra en muy pocas componentes. La Figura 11 presenta

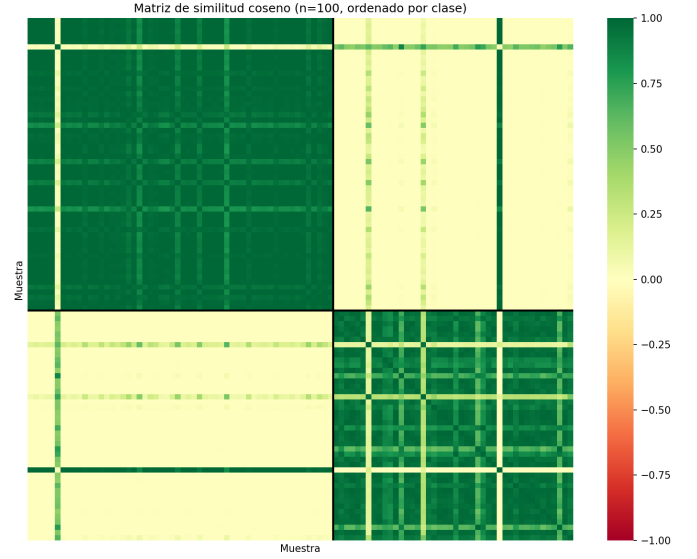


Fig. 9. Matriz de similitud coseno sobre 100 imágenes de test ordenadas por clase. Los dos bloques diagonales (intra-clase) son uniformemente cercanos a 1, mientras que los bloques fuera de la diagonal (inter-clase) son visiblemente más oscuros.

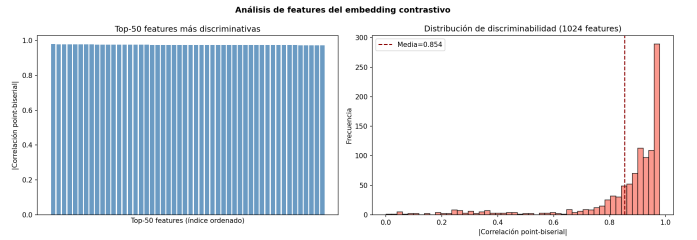


Fig. 10. *Score* de discriminabilidad (correlación punto-biserial) de las 50 dimensiones más informativas de los embeddings.

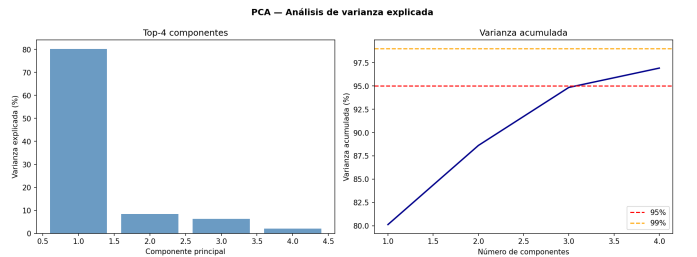


Fig. 11. Varianza explicada acumulada por componente principal. Las primeras 4 componentes capturan $\geq 95\%$ de la varianza total.

la curva de varianza explicada acumulada, mostrando que únicamente 4 componentes principales bastan para capturar el 95 % de la varianza total —una reducción efectiva del 99.6 % (1024 $\rightarrow$ 4).

La proyección UMAP a 2D revela dos clústeres bien separados, tanto en el conjunto de entrenamiento (Figura 12) como en el conjunto de prueba (Figura 13). La consistencia entre ambas figuras sugiere que la estructura geométrica del espacio aprendido es estable y generaliza más allá del conjunto de entrenamiento.

La comparación directa entre PCA y UMAP sobre el conjunto de test (Figura 14) revela diferencias cualitativas

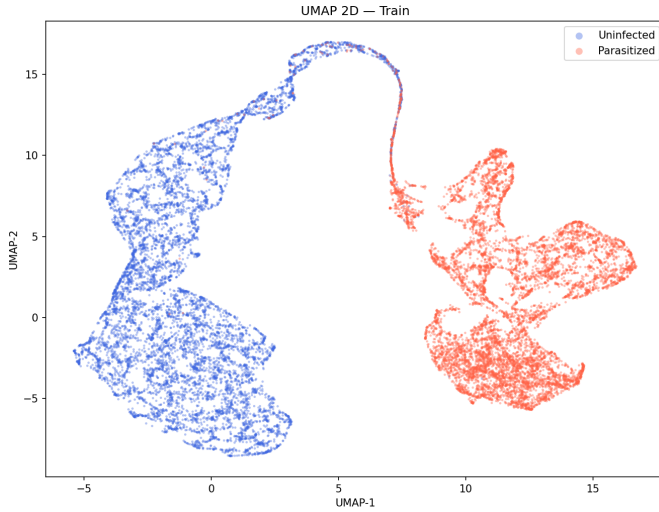


Fig. 12. Embeddings de *train* (19290 puntos) proyectados con UMAP ($n_{\text{neighbors}} = 15$, $\text{min_dist} = 0.1$, métrica coseno).

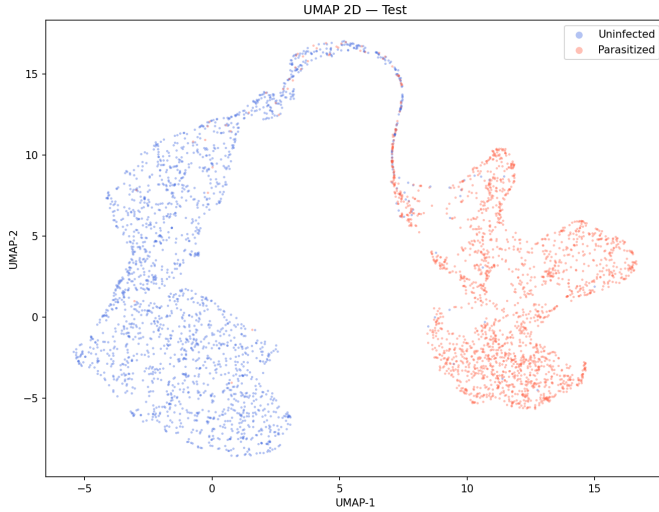


Fig. 13. Embeddings de *test* (4135 puntos) proyectados con UMAP. La estructura es consistente con la del conjunto de entrenamiento.

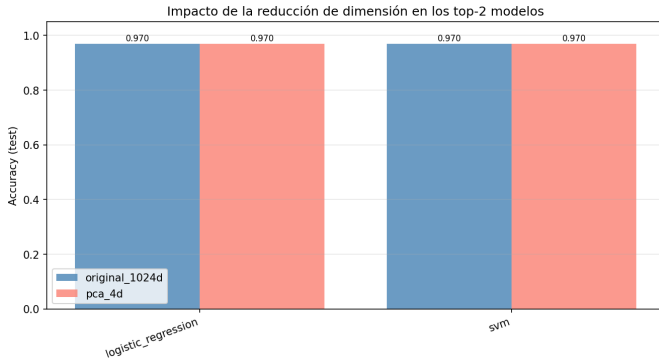


Fig. 14. Comparación lado a lado de PCA-2D vs. UMAP-2D sobre el test. UMAP genera clústeres más compactos y mejor separados.

significativas: UMAP genera clústeres más compactos y mejor separados que PCA, lo cual es esperable dado el carácter no

TABLE I
MÉTRICAS FINALES POR MODELO SOBRE EMBEDDINGS DE 1024 DIMENSIONES.

Modelo	Test Acc	F_1 -macro	ROC-AUC	BAcc
MLP	0.9703	0.9703	0.9932	0.9703
SVM (lineal)	0.9703	0.9703	n/a	0.9703
KNN ($k = 11$)	0.9700	0.9700	0.9852	0.9700
Regresión logística	0.9698	0.9698	0.9931	0.9698
Random Forest	0.9688	0.9688	0.9928	0.9688

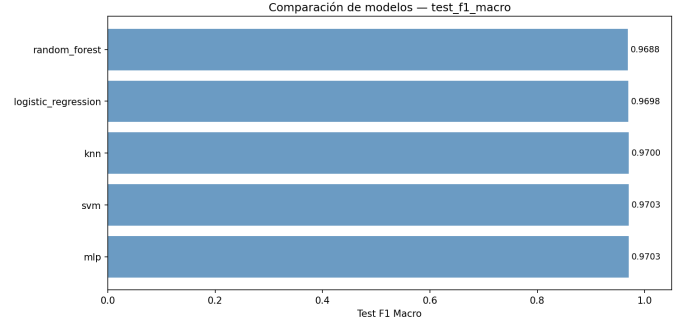


Fig. 15. F_1 -macro de test por modelo. El rango entre el mejor y el peor es de sólo 0.0015.

lineal y basado en vecindades del primero.

E. Modelos clásicos sobre embeddings de 1024 dimensiones

Los cinco modelos se ajustaron mediante GridSearchCV (3-fold estratificado, *scoring* f_1 -macro) sobre X_{trainval} y se evaluaron en X_{test} con IC bootstrap percentil al 95 % ($B = 200$). La Tabla I resume las principales métricas obtenidas.

La Figura 15 permite visualizar la diferencia entre modelos: el rango entre el mejor y el peor en F_1 -macro es de tan solo 0.0015, lo que indica que el espacio de embeddings aprendido por el encoder contrastivo es tan informativo que cualquiera de los cinco clasificadores alcanza un desempeño prácticamente equivalente.

Las curvas ROC superpuestas (Figura 16) corroboran lo anterior: los cuatro modelos con *scores* probabilísticos superan un AUC de 0.985, indicando una capacidad discriminativa prácticamente saturada.

Para un análisis granular del comportamiento por clase de cada clasificador, se examinan las matrices de confusión individuales sobre el conjunto de test ($N = 4135$). La Figura 17 presenta la matriz de la regresión logística, modelo cuya naturaleza lineal alcanza no obstante un desempeño cercano al óptimo, lo que sugiere que el espacio de embeddings es prácticamente linealmente separable.

La Figura 18 muestra el comportamiento de KNN ($k = 11$, métrica coseno). Aunque su AUC es el más bajo de los modelos probabilísticos, su accuracy es comparable, consistente con su naturaleza no paramétrica.

La matriz de Random Forest (Figura 19) revela el patrón de errores del modelo con mayor exactitud en entrenamiento (0.9948) y validación (0.9862) pero la menor exactitud en test (0.9688), evidenciando el sobreajuste más pronunciado del estudio (gap *train* $\rightarrow$ *test* del orden de -2.6 puntos porcentuales).

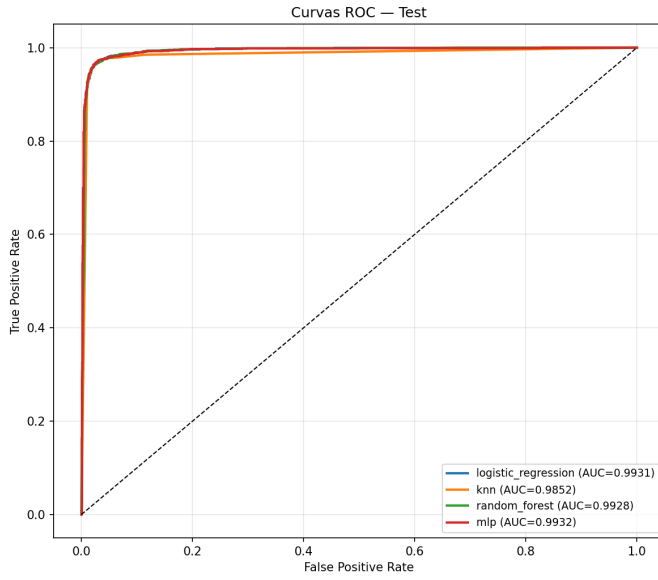


Fig. 16. Curvas ROC en el conjunto de test para los cuatro modelos con *scores* probabilísticos. Las áreas bajo la curva son superiores a 0.985 en todos los casos.

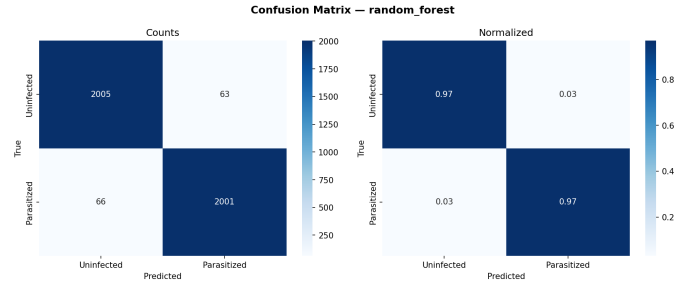


Fig. 19. Matriz de confusión — Random Forest.

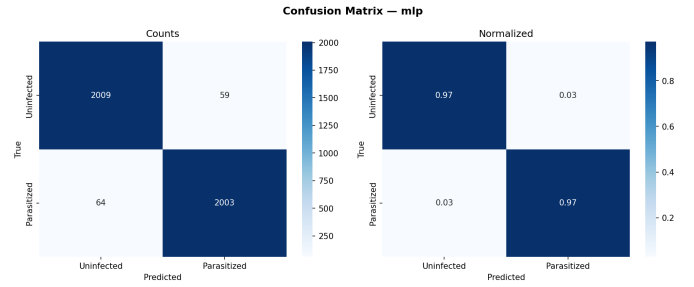


Fig. 20. Matriz de confusión — MLP.

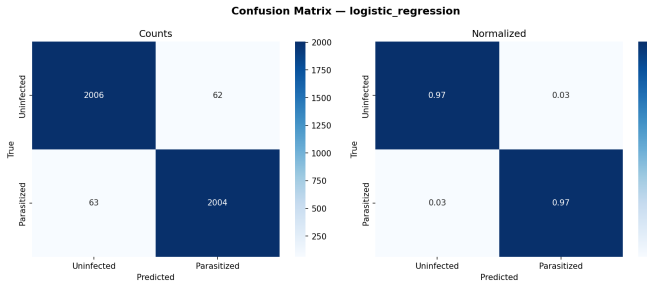


Fig. 17. Matriz de confusión — Regresión logística.

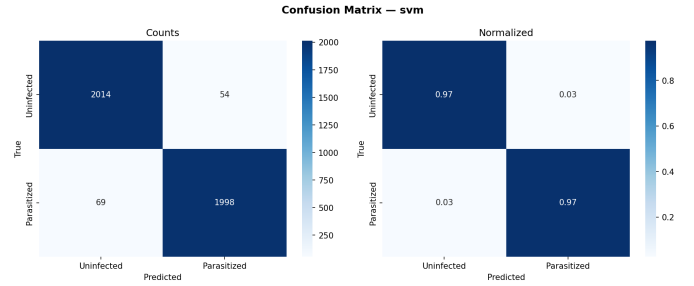


Fig. 21. Matriz de confusión — SVM (kernel lineal).

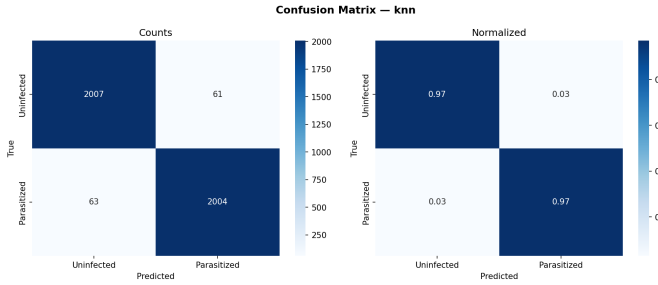


Fig. 18. Matriz de confusión — KNN.

El MLP (Figura 20) empata con SVM en *test* accuracy (0.9703) y F_1 -macro, pero obtiene el mejor AUC del estudio (0.9932) y el menor tiempo de inferencia (1.67 s), lo que lo posiciona como una opción atractiva en términos de balance entre precisión y eficiencia.

Finalmente, la matriz del SVM lineal (Figura 21) refuerza la observación de separabilidad lineal del espacio de embeddings: a pesar de no requerir un kernel no lineal, el modelo iguala al MLP en métricas globales, lo que es coherente con la geometría revelada por las proyecciones UMAP y los altos valores de discriminabilidad lineal por *feature*.

En conjunto, las matrices revelan que los errores son aproximadamente simétricos entre clases (entre 54 y 69 falsos en cada dirección), confirmando que ningún clasificador exhibe un sesgo sistemático hacia una clase.

F. Impacto de la reducción de dimensión

Para evaluar la robustez del espacio aprendido frente a una compresión extrema, los dos mejores modelos (regresión logística y SVM lineal) se reentrenaron sobre los embeddings proyectados a las 4 primeras componentes principales. Los resultados son notables: la regresión logística pasa de 0.9698 a 0.9703 en *test* accuracy (+0.05 pp), y el SVM se mantiene exactamente en 0.9703. No se observa degradación al pasar de 1024 a 4 dimensiones; la mejora marginal de la regresión logística podría atribuirse a un efecto de *denoising* implícito al descartar direcciones de varianza baja.

Este resultado refuerza la observación geométrica de la Subsección III-C: la información discriminativa entre clases está concentrada en muy pocas direcciones lineales del espacio de embeddings, validando que el encoder SupCon ha aprendido una representación intrínsecamente compacta del problema.

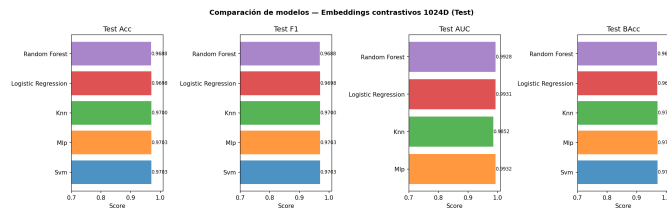


Fig. 22. Comparación final de Test Accuracy, F_1 -macro, ROC-AUC y Balanced accuracy entre los cinco modelos clásicos.

G. Análisis de vecinos más cercanos

Una inspección cualitativa de las consultas más cercanas (en métrica coseno) sobre el conjunto de test revela dos regímenes complementarios. En el caso típico, una imagen *Parasitized* recupera vecindarios homogéneos con similitudes superiores a 0.999 y todos los vecinos compartiendo etiqueta. En casos atípicos, una imagen *Parasitized* puede recuperar como vecinos más cercanos a cinco imágenes etiquetadas como *Uninfected*, también con similitudes superiores a 0.999. Este patrón es coherente con la existencia de casos *borderline* en el dataset y constituye una pista clara tanto para auditoría manual de etiquetas como para la incorporación de técnicas de *label noise correction* en trabajo futuro.

H. Comparación consolidada

La Figura 22 sintetiza los cuatro principales indicadores de calidad sobre el conjunto de test: *Test Accuracy*, F_1 -macro, ROC-AUC y *Balanced Accuracy*, para los cinco clasificadores.

Como síntesis cuantitativa: todos los modelos superan el 96.8% de accuracy y 0.985 de AUC; el rango entre el mejor y el peor modelo es ≤ 0.0015 en F_1 -macro y ≤ 0.0080 en AUC, con intervalos de confianza bootstrap que se solapan completamente; la reducción del espacio de 1024 a 4 dimensiones mediante PCA no degrada la performance de los mejores modelos; y el *gap* de similitud coseno intra/inter-clase es $\Delta = 0.8170$, evidencia geométrica directa de que el encoder SupCon construyó una representación discriminativa de alta calidad.

Estas observaciones, en conjunto, demuestran que (i) el aprendizaje contrastivo supervisado produce embeddings cuya capacidad de discriminación es prácticamente saturada por cualquier clasificador estadístico razonable, y (ii) la complejidad efectiva del problema, una vez transformado al espacio de embeddings, es muy baja —pocas dimensiones lineales bastan para alcanzar el rendimiento final.

REFERENCES

- [1] World Health Organization, “World malaria report 2024,” World Health Organization, Geneva, Tech. Rep., 2024.
- [2] J. Sora-Cardenas, W. M. Fong-Amaris, C. A. Salazar-Centeno, A. Castañeda, O. D. Martínez-Bernal, D. R. Suárez, and C. Martínez, “Image-based detection and classification of malaria parasites and leukocytes with quality assessment of romanowsky-stained blood smears,” *Sensors*, vol. 25, no. 2, p. 390, 2025.
- [3] A. M. Rojas Tocora, “Clasificación de parásitos de la malaria,” 2024.
- [4] P. S. Segovia, L. O. Cedeño, and L. Z. Gómez, “Diagnóstico de malaria mediante el uso de redes neuronales convolucionales,” 2021.
- [5] A. Ferreras Extremo *et al.*, “Estudio de algoritmos de redes neuronales convolucionales en dataset de imágenes médicas,” 2021.

- [6] M. F. Ahamed, M. Nahiduzzaman, G. Mahmud, F. B. Shafi, M. A. Ayari, A. Khandakar, M. Abdullah-Al-Wadud, and S. R. Islam, “Improving malaria diagnosis through interpretable customized cnns architectures,” *Scientific reports*, vol. 15, no. 1, p. 6484, 2025.
- [7] W. Siłka, M. Wiczorek, J. Siłka, and M. Woźniak, “Malaria detection using advanced deep learning architecture,” *Sensors*, vol. 23, no. 3, p. 1501, 2023.
- [8] A. M. Bautista, L. E. Ñ. Vargas, and E. M. Vilca, “Comparativa de modelos basados en redes neuronales convolucionales: Resnet-50v2, mobilenetv2 e efficientnetb0 en la detección de malaria,” *Micaela Revista de Investigación-UNAMBA*, vol. 5, no. 1, pp. 42–49, 2024.
- [9] S. Rajaraman, S. K. Antani, M. Poostchi, K. Silamut, M. A. Hossain, R. J. Maude, S. Jaeger, and G. R. Thoma, “Pre-trained convolutional neural networks as feature extractors toward improved malaria parasite detection in thin blood smear images,” *PeerJ*, vol. 6, p. e4568, 2018.
- [10] P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, and D. Krishnan, “Supervised contrastive learning,” *Advances in neural information processing systems*, vol. 33, pp. 18 661–18 673, 2020.
- [11] A. Jain, “Malaria cell images dataset,” <https://www.kaggle.com/datasets/iarunava/cell-images-for-detecting-malaria>, 2018.
- [12] C. Shorten and T. M. Khoshgoftaar, “A survey on image data augmentation for deep learning,” *Journal of Big Data*, vol. 6, no. 1, pp. 1–48, 2019.
- [13] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [14] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “Imagenet: A large-scale hierarchical image database,” in *2009 IEEE conference on computer vision and pattern recognition*. Ieee, 2009, pp. 248–255.
- [15] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A simple framework for contrastive learning of visual representations,” in *International Conference on Machine Learning*. PMLR, 2020, pp. 1597–1607.
- [16] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations*, 2019.
- [17] —, “SGDR: Stochastic gradient descent with warm restarts,” in *International Conference on Learning Representations*, 2017.
- [18] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” *Advances in Neural Information Processing Systems*, vol. 32, pp. 8024–8035, 2019.
- [19] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [20] T. Cover and P. Hart, “Nearest neighbor pattern classification,” *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.
- [21] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [22] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [23] L. McInnes, J. Healy, and J. Melville, “Umap: Uniform manifold approximation and projection for dimension reduction,” *arXiv preprint arXiv:1802.03426*, 2018.
- [24] T. Fawcett, “An introduction to ROC analysis,” *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.
- [25] B. Efron, “Bootstrap methods: Another look at the jackknife,” *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.